

Mastercard Innovation Challenge 2026

AI Defence Lab for Payment Security

Implemented First-Cut Architecture and Next-Step Options

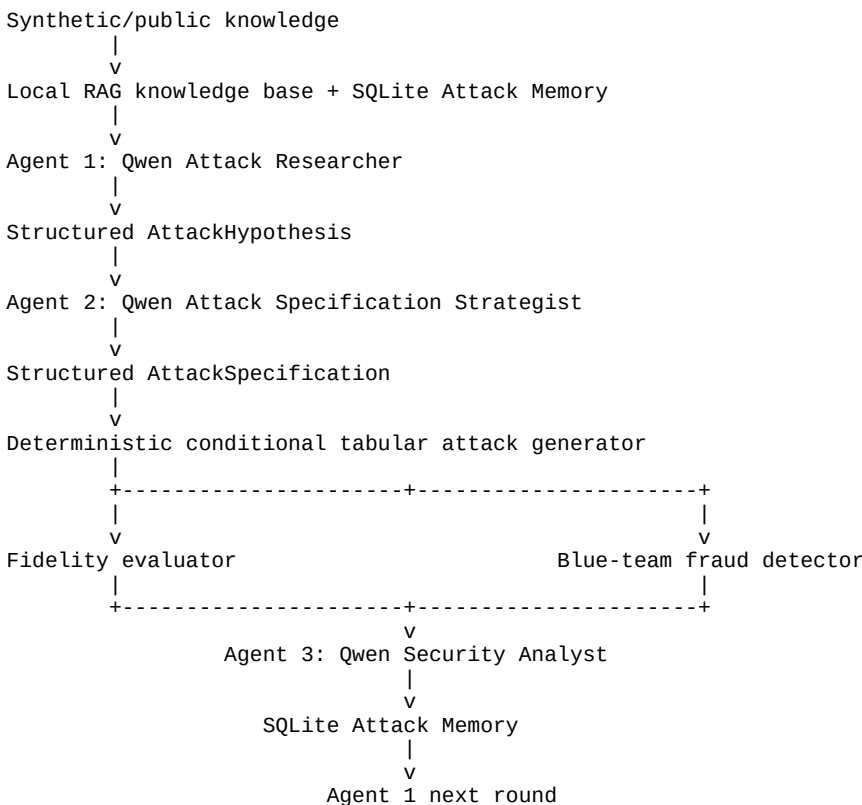
Status date: 22 August 2026

Executive Summary

A first-cut closed-loop red-team/blue-team prototype has been implemented and executed on a Kaggle GPU from VS Code. The system uses one local Qwen2.5-7B-Instruct GGUF Q4_K_M model across three logical roles: Attack Researcher, Attack Specification Strategist, and Security Analyst. The prototype completed two synthetic-data rounds with the required feedback direction: Agent 3 -> Attack Memory -> Agent 1.

The current result is an engineering baseline, not a final competition claim. The detector and loop are working; generator fidelity, attack diversity, novelty, public-research grounding, and web-prototype polish still need strengthening before submission.

Implemented Architecture



The implementation deliberately prevents direct Agent 3 -> Agent 2 feedback. Agent 2 receives the Agent 1 hypothesis and schema constraints only. Agent 3 findings are written to Attack Memory and become available to Agent 1 during the next research round.

Components Implemented

Shared local LLM

- Model: Qwen2.5-7B-Instruct GGUF, Q4_K_M.
- Current distribution: two GGUF shards totaling approximately 4.361 GiB.
- Storage: local models/ directory and private Kaggle Dataset; model files are excluded from GitHub.
- Runtime: llama-cpp-python CUDA wheel on Kaggle.
- Reuse: one lazily loaded model instance is shared across all three logical roles.
- Structured output: JSON response parsing, bounded output length, retry on malformed JSON, and Pydantic validation.

Agent 1: Attack Researcher

Consumes retrieved evidence, accumulated Attack Memory, and synthetic-scope constraints. Produces an AttackHypothesis with attack family, scenario, target context, behavioural mechanism, novelty rationale, research direction, evidence references, and memory context.

Agent 2: Attack Specification Strategist

Consumes only the Agent 1 hypothesis. Produces an AttackSpecification containing temporal, amount, device, beneficiary, realism, feature, and evasion constraints. It does not receive detector findings directly.

Agent 3: Security Analyst

Consumes detector metrics and fidelity evidence. Produces a WeaknessReport with observed weaknesses, supporting evidence, priority, confidence, and a recommended next attack direction. The report is persisted to Attack Memory.

RAG layer

The first iteration uses a local, curated text knowledge base with a synthetic seed document. Retrieval is a lightweight term-overlap mechanism that returns evidence excerpts and source identifiers. It is intentionally simple and auditable.

Attack Memory

SQLite stores hypotheses, specifications, evaluation summaries, and weakness reports. The next Agent 1 call retrieves recent records. This separates internal experiment experience from external/public knowledge.

Generator

The first cut is a deterministic conditional tabular baseline, not a GAN or diffusion model. It generates attack rows conditioned on the selected attack family and retains attack ID, family, round, method, and fraud label metadata.

Fidelity evaluator

The current evaluator reports amount mean delta, amount standard-deviation delta, fraud rate, and a heuristic behavioural-plausibility score. It is separate from detection so realism and detectability remain distinct objectives.

Detector

The blue team uses a scikit-learn pipeline with one-hot channel encoding and a HistGradientBoostingClassifier. It reports precision, recall, F1, ROC-AUC, false-positive rate, and confusion matrix on a holdout-plus-attack evaluation set.

Web prototype

A Streamlit entry point exposes the two-round demonstration and displays hypotheses, specifications, fidelity, detection, and Agent 3 findings.

Synthetic Data Used

No real cardholder data, PII, production payment data, confidential data, or live-system data was used.

The reference transaction generator creates 400 legitimate synthetic rows per configured run using a fixed seed. Each row contains:

- amount: lognormal synthetic amount.
- hour: integer hour from 0 to 23.
- device_change: binary synthetic signal.
- beneficiary_change: binary synthetic signal.
- velocity_24h: Poisson-distributed short-window transaction count.
- channel: one of web, mobile, or card_present.
- is_fraud: legitimate label 0.

Each round generates 80 synthetic attack rows. The baseline supports attack families including:

- low_and_slow: lower amounts and reduced transaction velocity.
- trusted_device: lower device-change frequency to test detector reliance on device novelty.
- Generic conditional attack rows with higher amount, device-change, beneficiary-change, and velocity distributions.

Generated records retain:

- attack ID,
- attack family,
- round number,
- generation method,
- ground-truth fraud label.

The current dataset is a controlled synthetic demonstration schema. It is not a claim that these distributions represent live Mastercard payment data.

Validation Evidence

The Kaggle-connected VS Code notebook verified:

- CUDA available: True.
- GPU: Tesla T4.
- GPU count: 2.
- Both Qwen GGUF shards accessible from the private Kaggle Dataset.
- Total model size: approximately 4.361 GiB.
- CUDA-enabled llama.cpp GPU offload support: True.
- Qwen structured JSON inference: passed.

- Two-round Qwen-backed loop: passed.
- Agent backend: QwenAgents.
- Detection F1 in the validated two-round run: 0.988 and 0.988.
- Feedback path: Agent 3 -> Memory -> Agent 1: OK.

The F1 values are results from the current synthetic baseline and should not be presented as official competition scores or real-world deployment performance.

Rules and Compliance Position

The supplied Rules snapshot requires a code repository, solution walkthrough, and working web prototype. It permits synthetic, anonymized, or authorized sample data and prohibits real cardholder data, PII, production payment data, live-system targeting, payment-infrastructure targeting, and third-party targeting.

The current implementation follows these boundaries:

- Synthetic-only data in iteration 1.
- Offline attack generation and evaluation.
- No live payment-system interaction.
- Model weights kept outside GitHub in a private Kaggle Dataset.
- Public research sources for iteration 2 must be reviewed for authorization, licensing, confidentiality, and provenance before ingestion.
- The Streamlit prototype and GitHub repository are present as implementation artifacts.

Logical Next-Step Options

Option A: Submission-hardening baseline

Recommended first. Keep the deterministic generator, but improve evaluation and reproducibility:

1. Add explicit train, generator-fit, detector-train, validation, and unseen-test splits.
2. Add per-attack-family precision, recall, F1, PR-AUC, ROC-AUC, calibration, and false-positive analysis.
3. Add attack diversity, deduplication, novelty, and round-over-round reports.
4. Save every configuration, prompt, model identifier, seed, and output artifact.
5. Make the Streamlit demo show the two-round feedback change clearly.

****Benefit:**** highest confidence and lowest compute risk before submission.

Option B: Public-research RAG upgrade

Recommended for iteration 2. Add a small, reviewed corpus of public fraud research, security reports, typologies, and permitted documentation.

1. Record source URL, title, publisher, date, license, and retrieval date.
2. Chunk documents with stable source IDs.
3. Replace term overlap with embedding retrieval and optional reranking.
4. Require evidence references in Agent 1 output.

5. Add a source-review manifest so no confidential or unauthorized material enters the corpus.

****Benefit:**** stronger identification breadth, evidence grounding, and novelty rationale.

Option C: Stronger tabular generator

Use only after Option A is stable and only in Kaggle GPU sessions:

1. Establish a baseline against the deterministic generator.
2. Try a conditional tabular model such as CTGAN or a tabular diffusion implementation.
3. Fit only on permitted synthetic/reference data.
4. Keep the final detector evaluation set unseen by generator fitting.
5. Compare marginal distributions, dependencies, conditional behaviour, downstream detector utility, and privacy/leakage sanity checks.

****Benefit:**** improved fidelity and diversity; higher runtime, debugging, and evaluation risk.

Option D: Detector robustness upgrade

Add stronger features or models only if baseline evidence justifies it:

- threshold tuning for false-positive control,
- calibrated probabilities,
- temporal/velocity aggregates,
- sequence or graph features if the schema expands,
- model comparison with XGBoost or LightGBM.

****Benefit:**** stronger defend pillar; risk of overfitting synthetic artifacts if evaluation splits are weak.

Recommended Order

1. Freeze and document the validated two-round Qwen baseline.
2. Add reproducible split management and complete evaluation reports.
3. Upgrade RAG with reviewed public sources for iteration 2.
4. Add attack-family diversity and novelty tracking.
5. Compare a stronger conditional generator against the baseline on Kaggle GPU.
6. Improve the Streamlit walkthrough and package the code repository plus solution document.
7. Run a final compliance, provenance, privacy, and reproducibility audit before submission.

Current Limitations

- The generator is deterministic and simple; it is not yet CTGAN, diffusion, or adversarial training.
- The RAG corpus is a seed document, not a broad public research collection.
- The fidelity score is heuristic and not an official Mastercard scoring formula.
- The current synthetic distributions are not evidence of live-payment behaviour.

- The validated two-round run demonstrates engineering closure, not production readiness.
- More comprehensive unseen evaluation and attack novelty analysis are required before making strong competition claims.

Final Position

The project has a working, competition-aligned first cut: Identify through RAG and memory, Generate through structured specifications and synthetic tabular generation, and Defend through a measurable detector. The most defensible next move is to strengthen evaluation and evidence provenance before increasing model complexity.